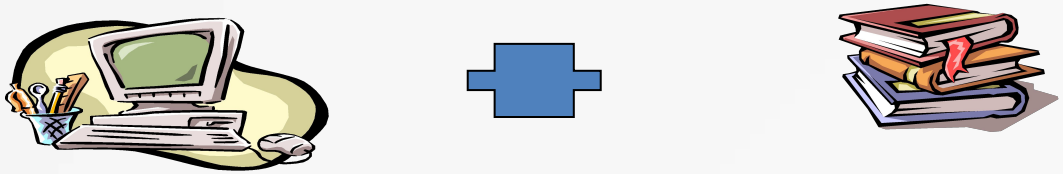
SOFTWARE ENGINEERING and DevOps

CSE2605

Sonali R. Dhave
Assistant Professor
FCI

UNIT I - INTRODUCTION:

- Computer programs and associated documentation



• Software is more than just a program code.

• A program is an executable code, which servers some computational purpose.

• Software is the collection of computer programs, procedures rules and associated documentation and data.

• Software is an information transformer producing, managing, modifying, displaying or transforming information that can simple as a single bit or a complex as a multimedia application.

Software Products:

- Software products may be developed for a particular customer or may be developed for a general market.
- **Software products may be:**
  - Generic- developed to be sold to a range of different customers
  - Bespoke (custom) - developed for a single customer according to their specification

What are the attributes of good software?

- Maintainability.
- Dependability
- Efficiency
- Usability

What is the difference between software engineering and computer science?

Computer Science	Software Engineering
is concerned with	
➢ theory ➢ fundamentals Algorithms, date structures, complexity theory, numerical methods	➢ the practicalities of developing ➢ delivering useful software SE deals with practical problems in complex software products

Computer science theories are currently insufficient to act as a complete underpinning for software engineering, BUT it is a foundation for practical aspects of software engineering

Software Engineering Paradigms:

Software Characteristics:

- Software is developed or engineered, it is not manufactured in the classical sence.
- Software doesn’t “wear out”.
- Although the industry is moving towards component based assembly, most software continues to be custom to built.

Software Applications Types:

- System Software
- Real-time Software.
- Business Software.
- Engineering and Scientific Software.
- Embedded Software.
- Personal Computer Software.
- Web-based Software.
- Artifical Intelligence Software.

System Software-

System software—a collection of programs written to service other programs. Some system software (e.g., compilers, editors, and file management utilities) processes complex, but determinate,4 information structures. Other systems applications (e.g., operating system components, drivers, networking software, telecommunications processors) process largely indeterminate data.

Application software/ Real time Software-
stand-alone programs that solve a specific business need. Applications in this area process business or technical data in a way that facilitates business operations or management/technical decision making.

Engineering/scientific software—has been characterized by “number crunching” algorithms. Applications range from astronomy to volcanology, from automotive stress analysis to space shuttle orbital dynamics, and from molecular biology to automated manufacturing.

Embedded software—resides within a product or system and is used to implement and control features and functions for the end user and for the system itself

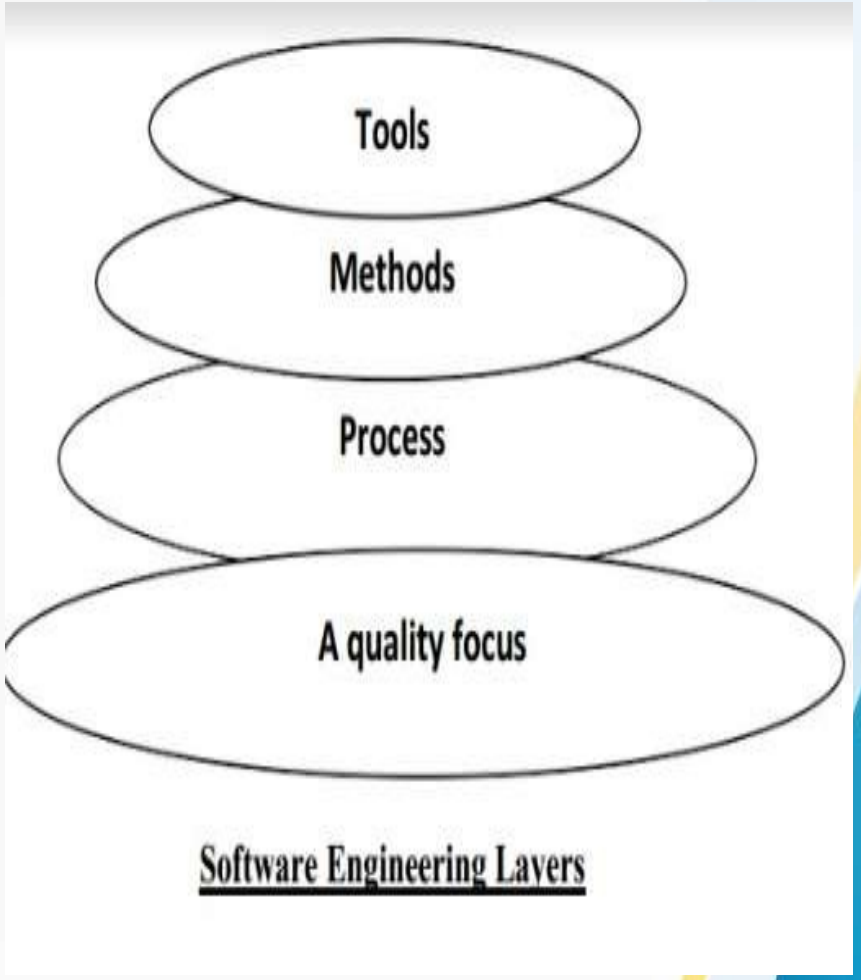
Product-line software—designed to provide a specific capability for use by many different customers. Product-line software can focus on a limited and esoteric marketplace (e.g., inventory control products) or address mass consumer markets (e.g., word processing, spreadsheets, computer graphics, multimedia, entertainment, database management, and personal and business financial applications)

Web applications—called “WebApps,” this network-centric software cate gory spans a wide array of applications. In their simplest form, WebApps can be little more than a set of linked hypertext files that present information using text and limited graphics

Artificial intelligence software—makes use of nonnumerical algorithms to solve complex problems that are not amenable to computation or straightfor ward analysis. Applications within this area include robotics, expert systems, pattern recognition (image and voice), artificial neural networks, theorem proving, and game playing

Software Engineering -A layered Technology:

- Application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software that is, the application of engineering software.



- The foundation for software engineering is the process layer. The software engi neering process is the glue that holds the technology layers together and enables rational and timely development of computer software. Process defines a framework that must be established for effective delivery of software engineering technology. The software process forms the basis for management control of software projects and establishes the context in which technical methods are applied, work products (models, documents, data, reports, forms, etc.) are produced, milestones are established, quality is ensured, and change is properly managed.

- Software engineering methods provide the technical how-to's for building soft ware. Methods encompass a broad array of tasks that include communication, requirements analysis, design modeling, program construction, testing, and sup port. Software engineering methods rely on a set of basic principles that govern each area of the technology and include modeling activities and other descriptive techniques.

- Software engineering tools provide automated or semiautomated support for the process and the methods. When tools are integrated so that information created by one tool can be used by another, a system for the support of software development, called computer-aided software engineering, is established.

The Software Process

A process is a collection of activities, actions, and tasks that are performed when some work product is to be created. An activity strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied. An action (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., an architectural design model). A task focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome

A process framework establishes the foundation for a complete software engi neering process by identifying a small number of framework activities that are appli cable to all software projects, s, regardless of their size or complexity. In addition, the process framework encompasses a set of umbrella activities that are applicable across the entire software process.

The five generic process framework activities

- The following generic process framework is applicable to the majority of software projects.
- Communication
- Planning
- Modelling
- Construction
- Deployment

Communication- Before any technical work can commence, it is critically important to communicate and collaborate with the customer. The intent is to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.

Planning- Any complicated journey can be simplified if a map exists. A software project is a complicated journey, and the planning activity creates a "map" that helps guide the team as it makes the journey. The map—called a software project plan—defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule.

Modeling- A software engineer does the same thing by creating models to better understand software requirements and the design that will achieve those requirements. Construction. This activity combines code generation (either manual or automated) and the testing that is required to uncover errors in the code.

Deployment- The software (as a complete entity or as a partially completed increment) is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation.

Umbrella Activities:

In general, umbrella activities are applied throughout a soft ware project and help a software team manage and control progress, quality, change, and risk.
Umbrella activities occur throughout the software process and focus primarily on project management, tracking, and control.

It includes:-

1. Software project tracking and control—allows the software team to assess progress against the project plan and take any necessary action to maintain the schedule.

2. Risk management—assesses risks that may affect the outcome of the project or the quality of the product. Software quality assurance—defines and conducts the activities required to ensure software quality.

3. Technical reviews—assesses software engineering work products in an effort to uncover and remove errors before they are propagated to the next activity.

4. Measurement—defines and collects process, project, and product measures that assist the team in delivering software that meets stakeholders' needs; can be used in conjunction with all other framework and umbrella activities.

5. Software configuration management—manages the effects of change throughout the software process.

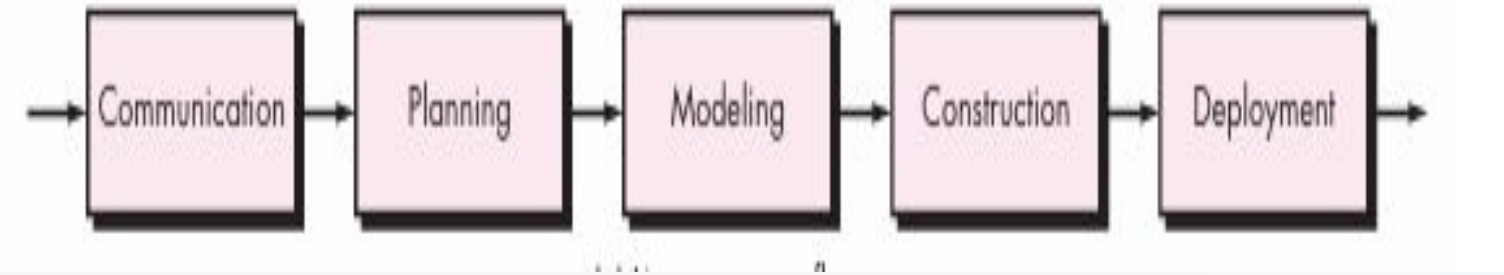
6. Reusability management—defines criteria for work product reuse (including software components) and establishes mechanisms to achieve reusable components.

7. Work product preparation and production—encompasses the activities required to create work products such as models, documents, logs, forms, and lists.

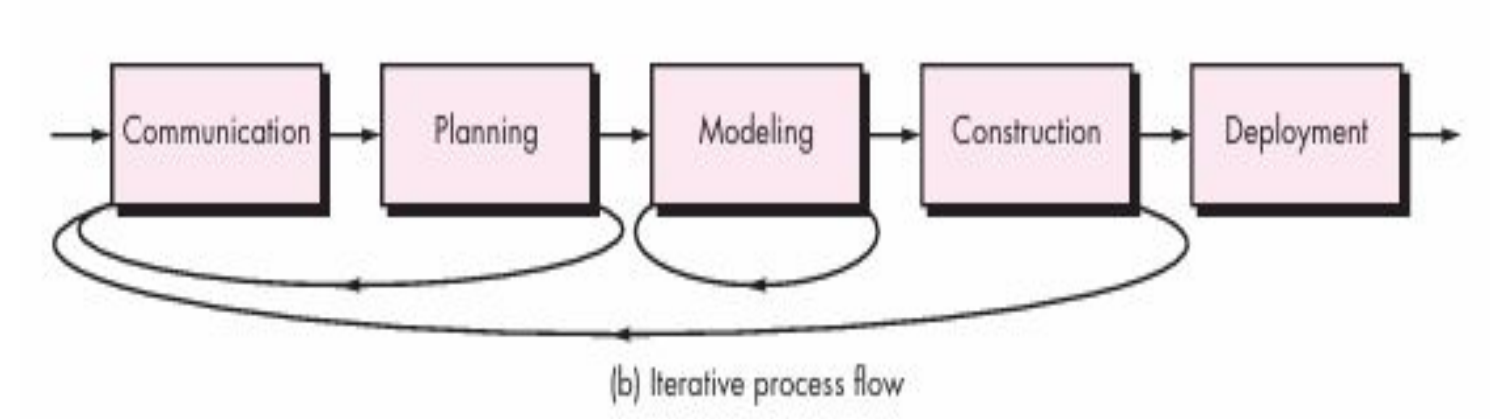
Process Flow-

Describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time.

1. Linear Flow- A linear process flow executes each of the five framework activities in sequence, beginning with communication and culminating with deployment

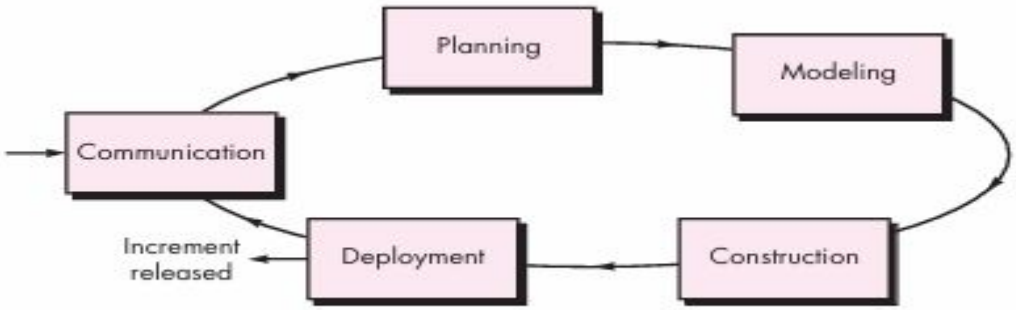


2. Iterative Flow- An iterative process flow repeats one or more of the activities before proceeding to the next .



3. Evolutionary flow-

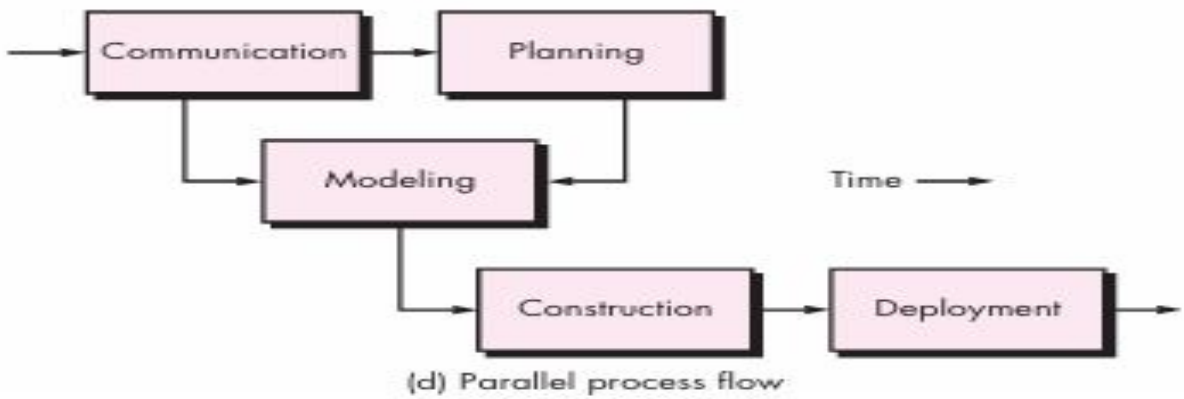
An evolutionary process flow executes the activities in a “circular” manner.



(c) Evolutionary process flow

4. Parallel flow-

A parallel process flow executes one or more activities in parallel with other activities.



(d) Parallel process flow

Software Development Life Cycle (SDLC)-

A software life cycle model (also termed process model) is a pictorial and diagrammatic representation of the software life cycle. A life cycle model represents all the methods required to make a software product transit through its life cycle stages. It also captures the structure in which these methods are to be undertaken.

Stage1: Planning and requirement analysis Requirement Analysis is the most important and necessary stage in SDLC. The senior members of the team perform it with inputs from all the stakeholders and domain experts or SMEs in the industry. Planning for the quality assurance requirements and identifications of the risks associated with the projects is also done at this stage.

Stage2: Defining Requirements Once the requirement analysis is done, the next stage is to certainly represent and document the software requirements and get them accepted from the project stakeholders. This is accomplished through "SRS"- Software Requirement Specification document which contains all the product requirements to be constructed and developed during the project life cycle.

Stage3: Designing the Software The next phase is about to bring down all the knowledge of requirements, analysis, and design of the software project. This phase is the product of the last two, like inputs from the customer and requirement gathering.

Stage4: Developing the project In this phase of SDLC, the actual development begins, and the programming is built. The implementation of design begins concerning writing code. Developers have to follow the coding guidelines described by their management and programming tools like compilers, interpreters, debuggers, etc. are used to develop and implement the code.

Stage4: Developing the project In this phase of SDLC, the actual development begins, and the programming is built. The implementation of design begins concerning writing code. Developers have to follow the coding guidelines described by their management and programming tools like compilers, interpreters, debuggers, etc. are used to develop and implement the code.

Stage5: Testing After the code is generated, it is tested against the requirements to make sure that the products are solving the needs addressed and gathered during the requirements stage. During this stage, unit testing, integration testing, system testing, acceptance testing are done.

Stage6: Deployment Once the software is certified, and no bugs or errors are stated, then it is deployed. Then based on the assessment, the software may be released as it is or with suggested enhancement in the object segment. After the software is deployed, then its maintenance begins.

Stage7: Maintenance Once when the client starts using the developed systems, then the real issues come up and requirements to be solved from time to time. This procedure where the care is taken for the developed product is known as maintenance.

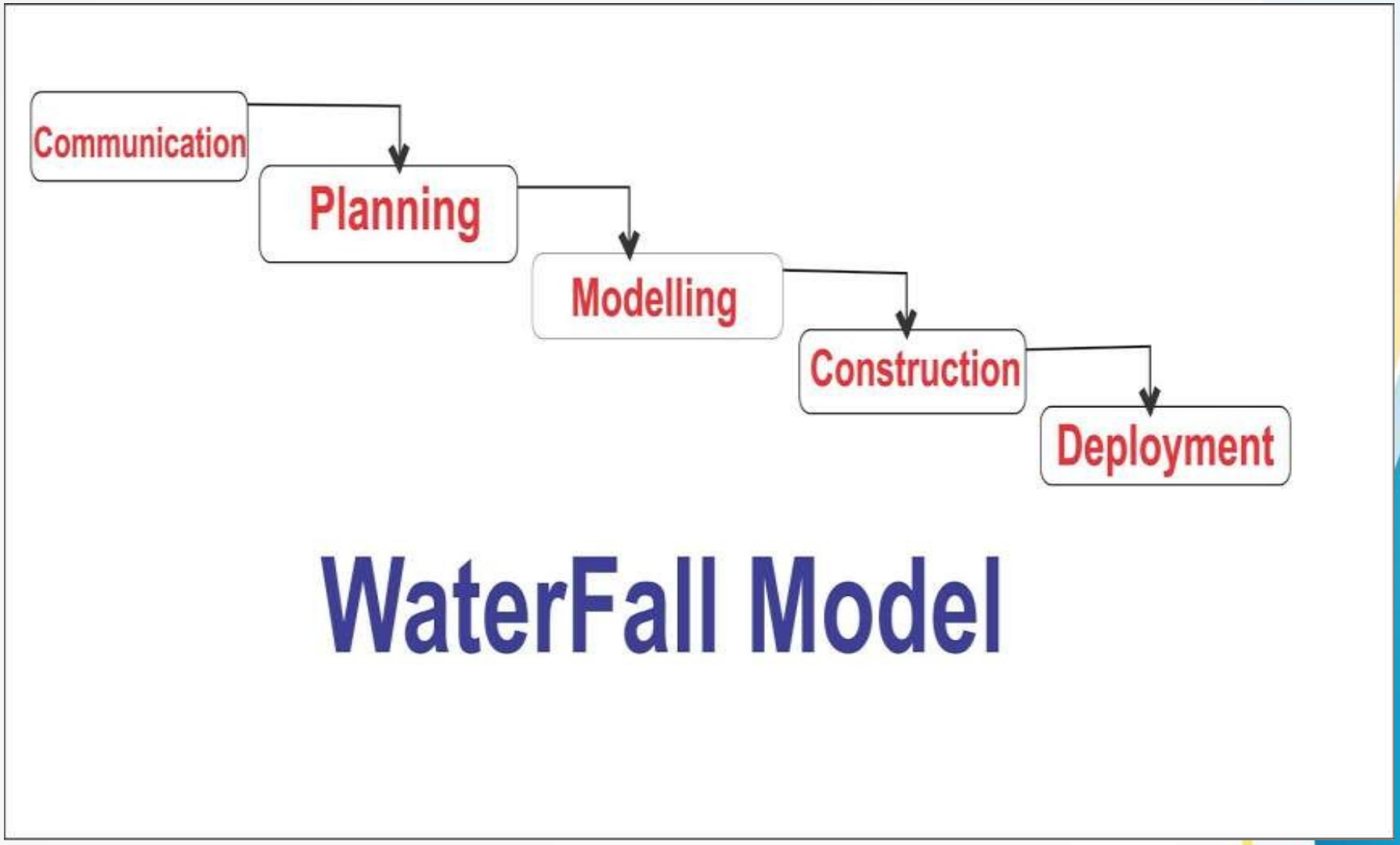
Process Models:

- Every software engineering organization should describe a unique set of framework activities for the software process it adopts.
    - Waterfall Life Cycle Model.
    - V Model
    - Iterative Waterfall Life Cycle Model.
    - Prototyping Model.
    - Incremental Model.
    - Sprial Model.
    - RAD Model.

Waterfall Life Cycle Model.

- Proposed by Winston Royce.
- It is called classic life cycle or Linear model.
- Requirements are well defined and stable.
- It suggests a systematic, sequential approach to software development.
- It begins with customer specification of requirements and progresses.
    - Planning.
    - Modeling.
    - Construction and
    - Deployment.

There are times when the requirements for a problem are well understood when work flows from communication through deployment in a reasonably linear fashion.



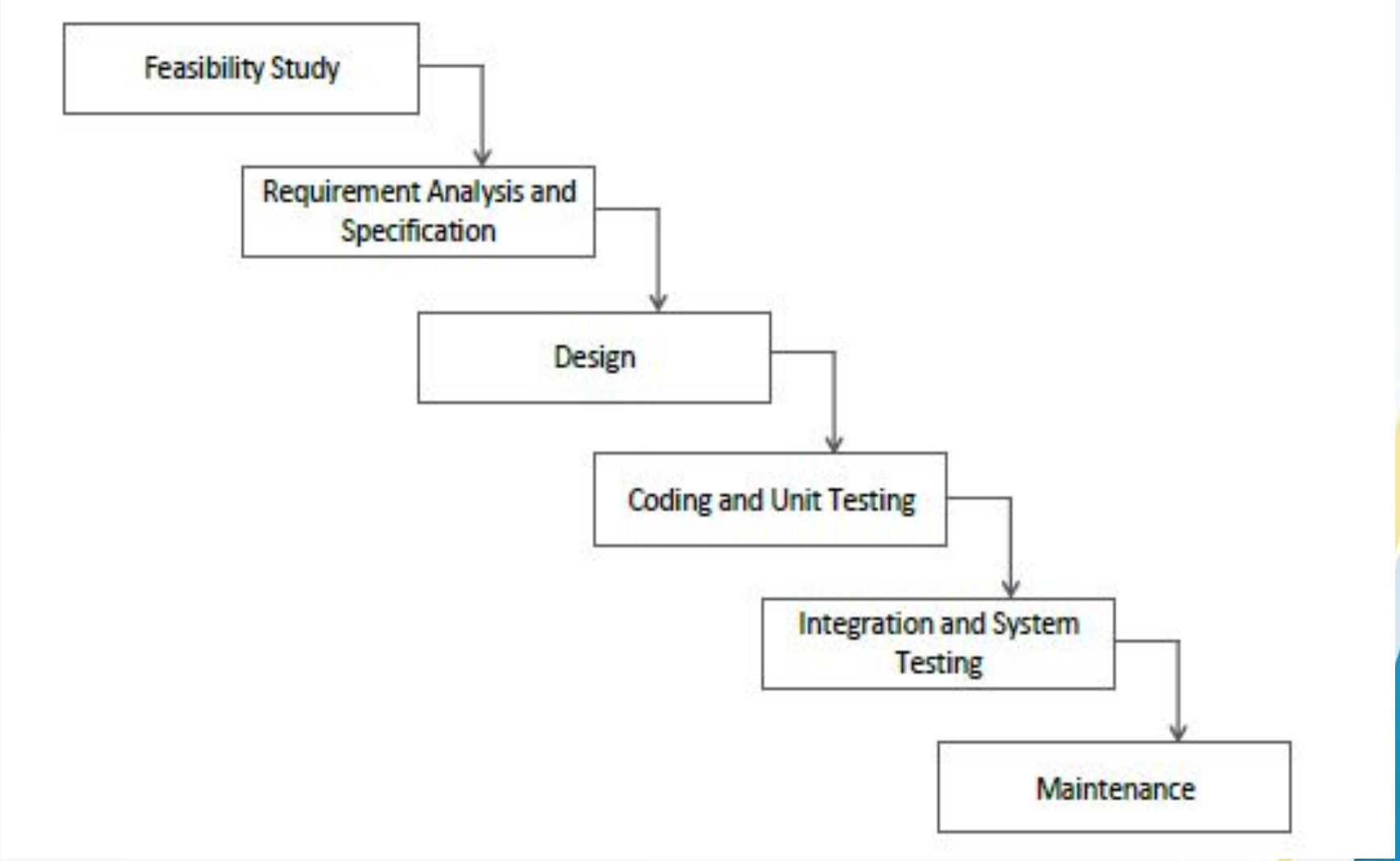
Advantages:

- Easy to understand.
- Each phase has well defined input and output.
- Helps project manger in proper planning of project.
- Provides a templates into which methods of analysis, design, code and support can be placed.

Disadvantages:

- One way street.
- It lack overlapping and interactions among phases.
- Model doesn't support delivery of system in pieces.

Phases of the Classical Waterfall Model:



Feasibility Study:

- It involves analysis of the problem and collection of allrelevant information relating to the product.
- The collected data are analysed.
  - Requriments of the Customer.
  - Formulations of the different strategies for solving the problem.
  - Evaluation of different solution strategies.

Requriments Analysis and Specification:

- It is understand the exact requriments of the customer and to document them properly.
  - Requirements gathering and analysis.
  - Requirements specification.

Design:

- The deign phase is to transform the requirements specified in the document into a structure that is suitable for implementation in some programming languaage.
  - Traditional Design Approach.
  - Object-Oriented Design Approach.

Coding and Unit Testing:

- The purpose mof the coding and unit testing phase of software development is to translate the software design into source code.

Integration and System Testing:

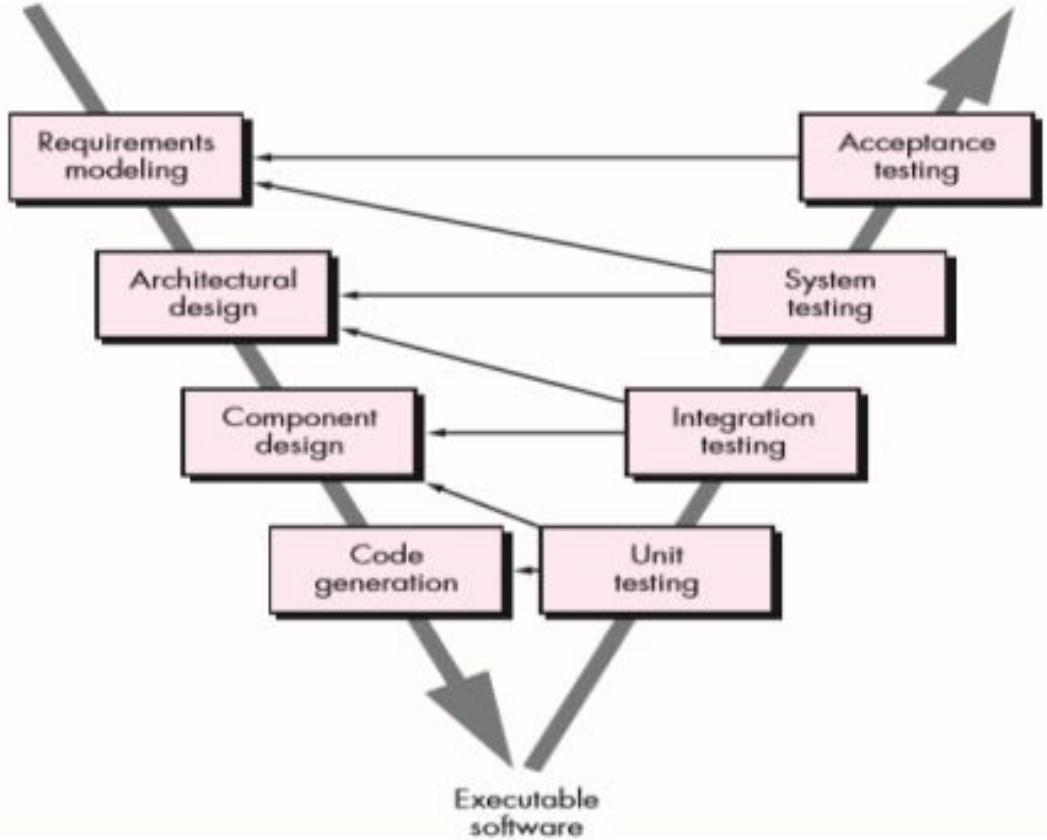
- ‘Integration of different modules is coded and unit tested.
  - $\alpha - \quad Testing$
  - $\beta - \quad Testing$
  - Accsptance Testing.

Maintenance:

- Maintenance of a typical software products requires much more than the effort necessary to develope the product itself.

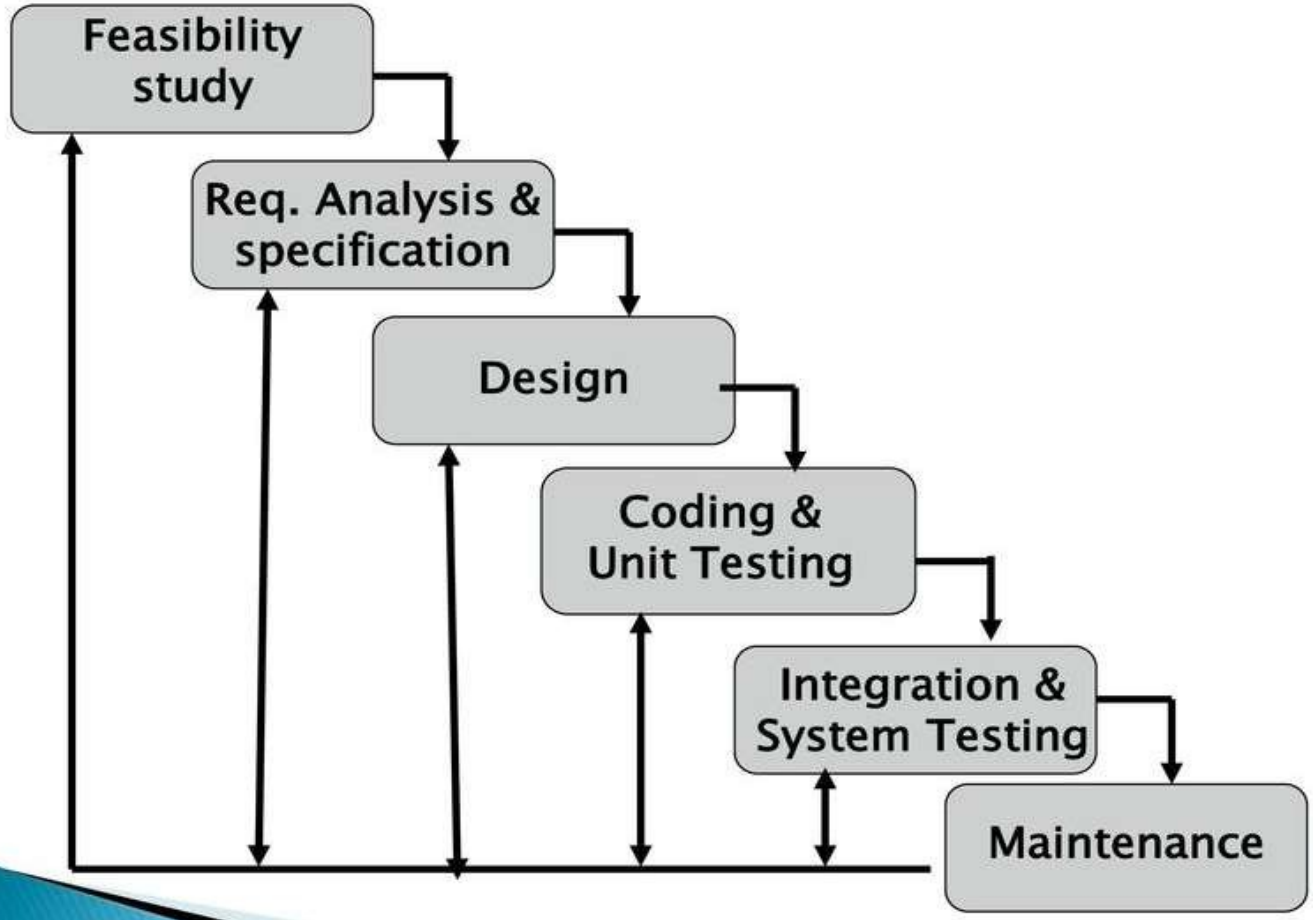
A variation in the representation of the waterfall model is called the V-model. As a software team moves down the left side of the V, basic problem requirements are refined into progressively more detailed and technical representations of the problem and its solution. Once code has been generated, the team moves up the right side of the V, essentially performing a series of tests (quality assurance actions) that validate each of the models created as the team moved down the left side.

FIGURE 2.4
The V-model



Iterative Waterfall life cycle model:

- The main changes is done by providing feedback paths from every phase to its preceding phase.



Prototype Model:

- Prototyping Model is a software development model in which prototype is tested, buil reworked until ant, acceptable and prototype is achieved.

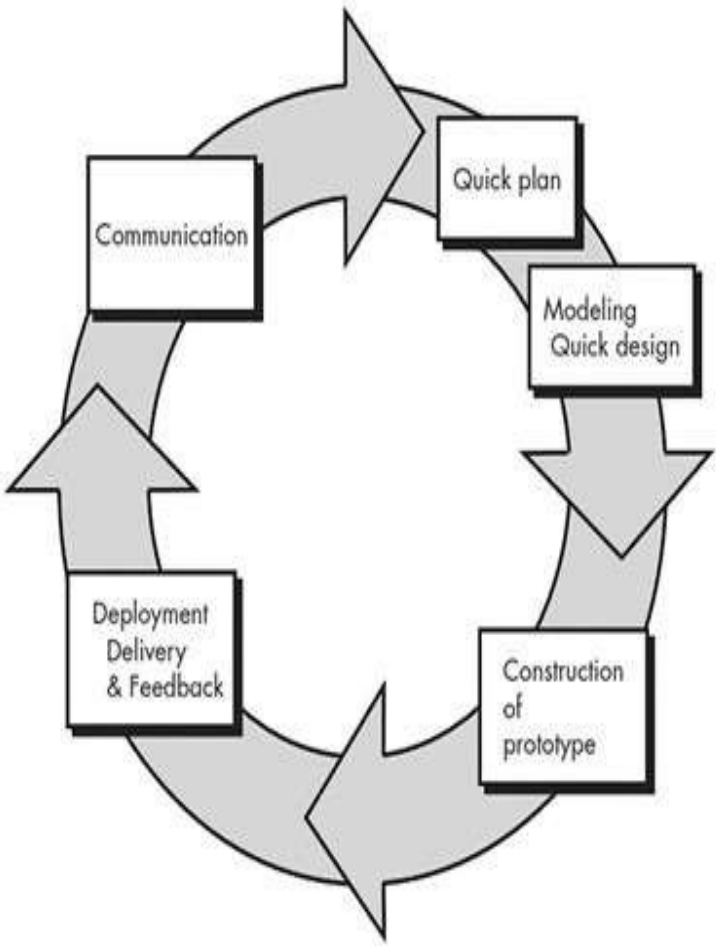


Figure: Prototype Model

Advantages:

- Clarity.
- Risk Identification.
- Good Environment.
- Take less time to complete.

Disadvantages:

- High cost.
- Slow process.
- Too many changes.

RAD Model:

- Rapid Application Development(RAD) is an incremental software model that a short development cycle.
- The RAD model is a “high-speed” of the waterfall model.
- The RAD process enables a development team to create a fully functional system within a very short time period.

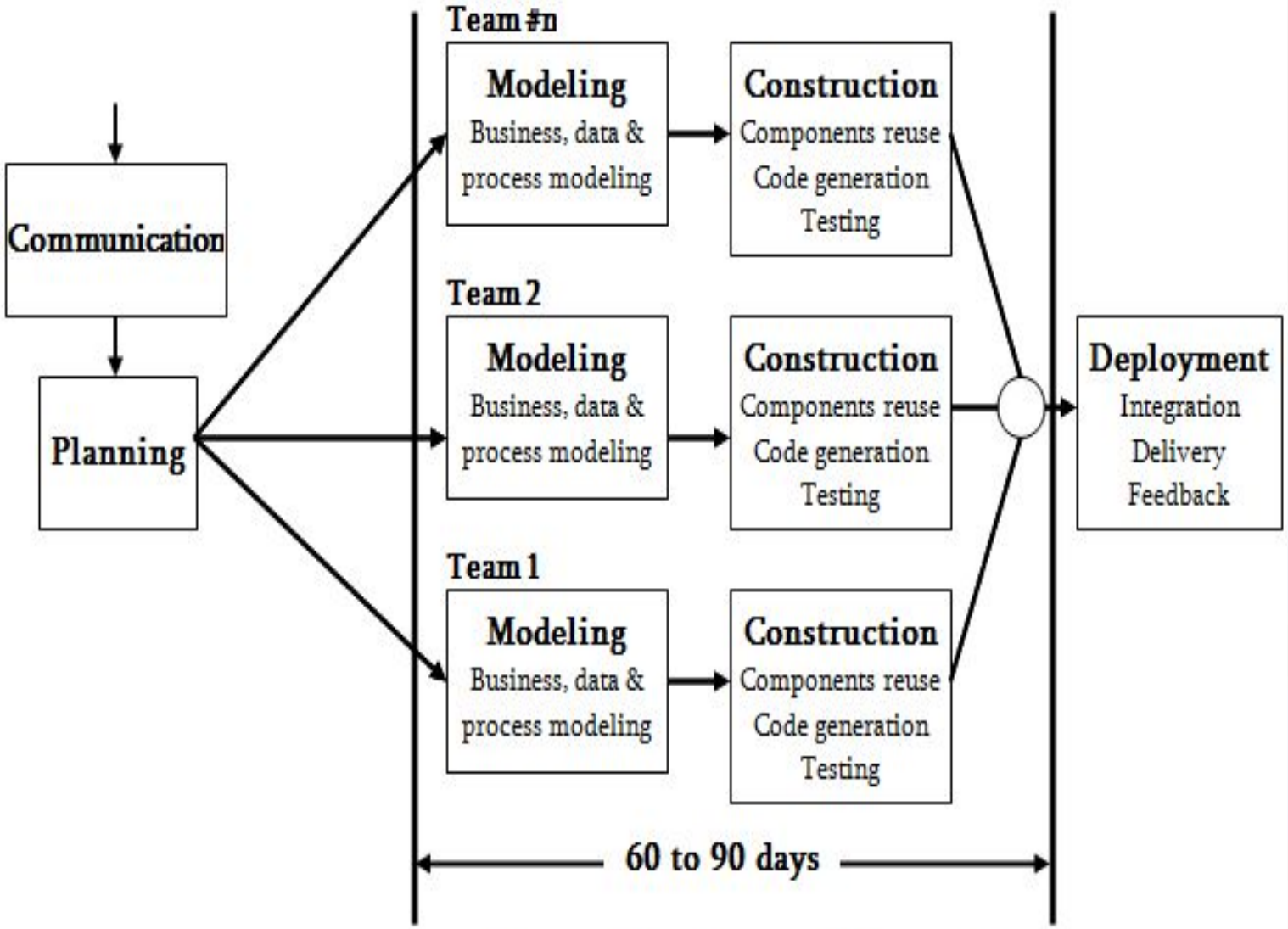


Figure : Flowchart of RAD model

Contents of RAD Pakages:

- Graphical user development environment.
- Reusable Components.
- Code generator.
- Programming Language.

Advantages:

- Fast products.
- Efficient Documentation.
- Interaction with user.

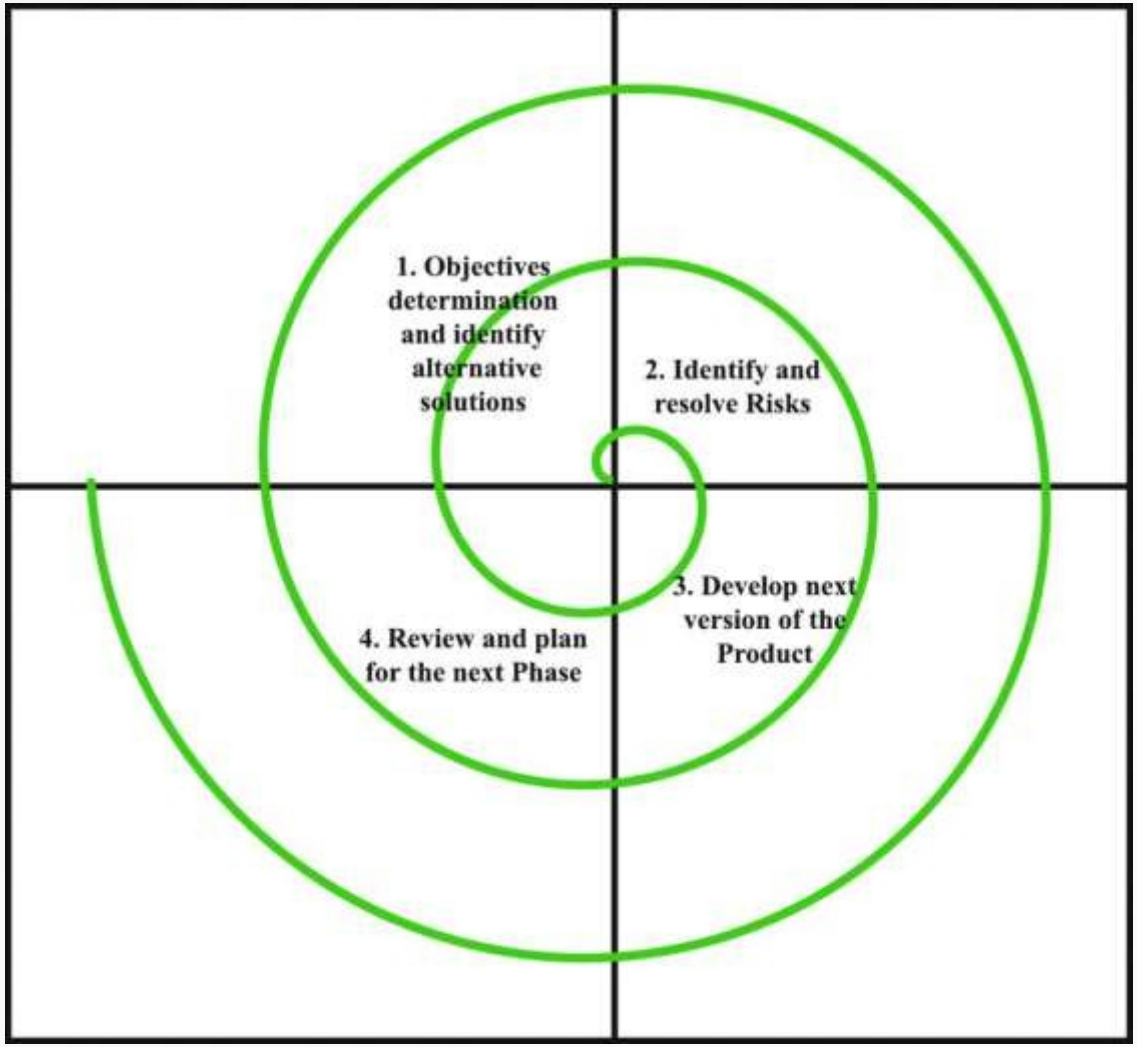
Disadvantages:

- User may not like fast activities.
- Not suitable for technical risks.

Sprial Model :

- This Spiral model is a combination of iterative development process model and sequential linear development model i.e. the waterfall model with a very high emphasis on risk analysis.
- The spiral model has four phases: Planning, Design, Construct and Evaluation.

Quadrants in sprial model :



Advantages :

- Risk Identification at early stage.
- Suitable for high rk projects.
- Flexibility for adding functionaility.

Disadvantages:

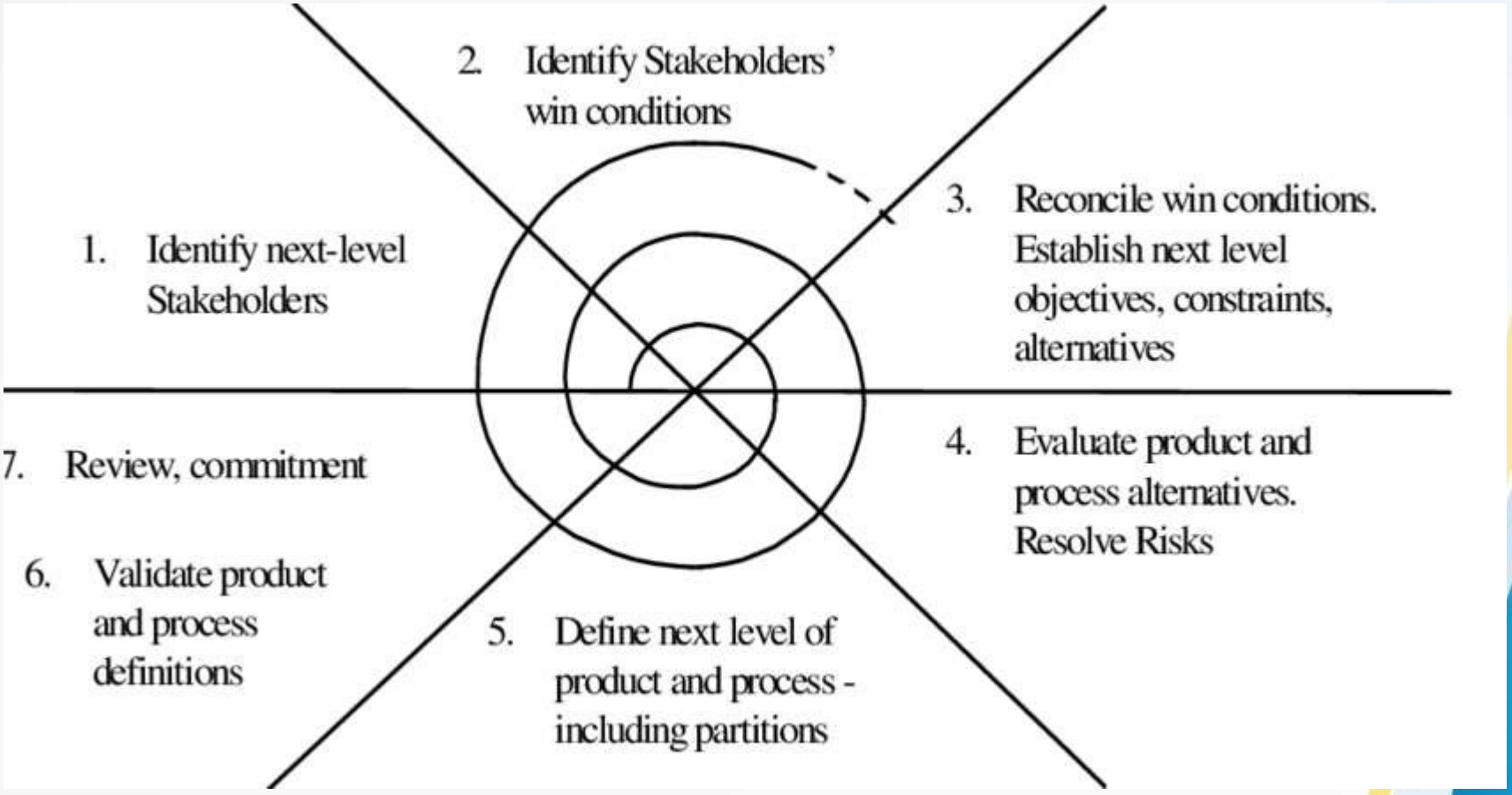
- Costly.
- Risk dependent.
- Not suitable for smaller projects.
- Difficult to meeting budget.

Win-Win Sprial Model:

- The customer wins by getting the system satisfying most of thier requirements and developers winsby working on achievable budgets and deadlines.
- **Advantages:**
- Lieghtweight methods suit small-medium size project.
- Produces good team.
- Test based approach to requirements and quality assurance

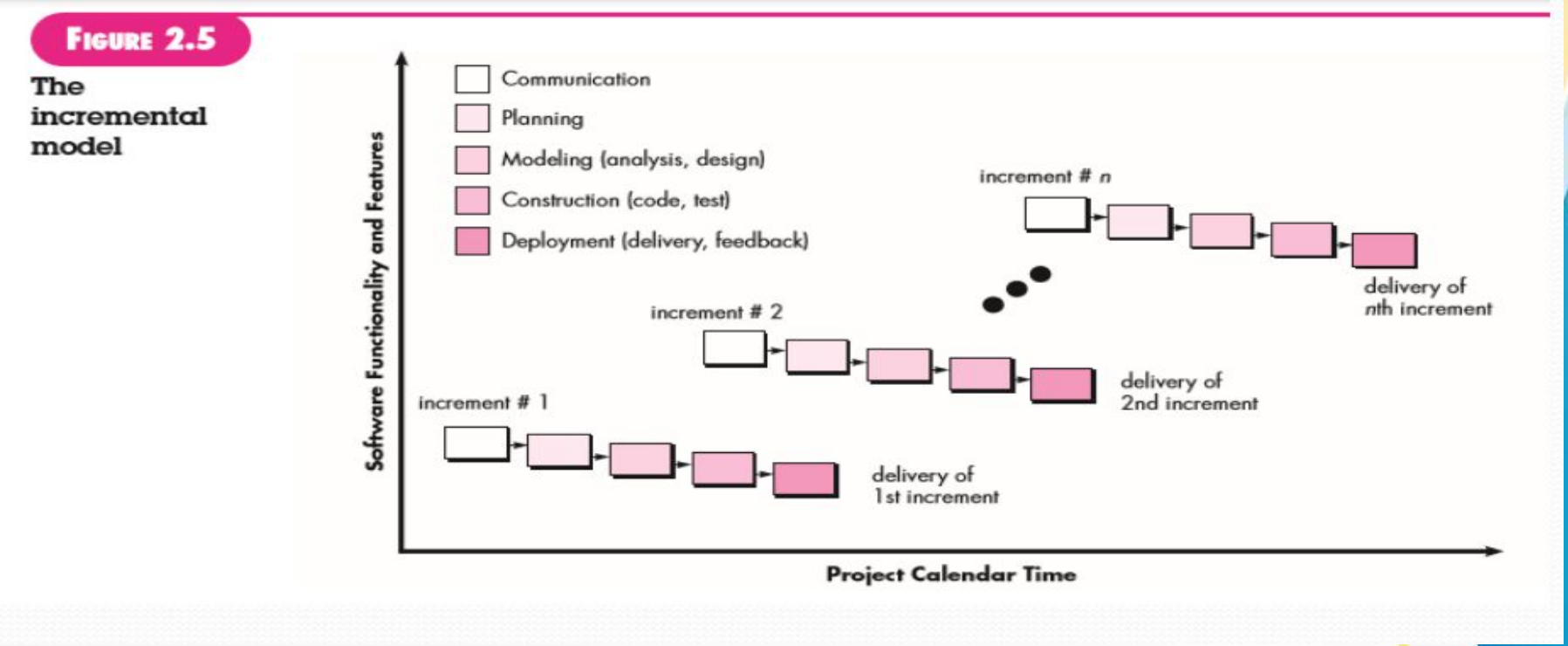
Diadvantages:

- Programming pairs is costly.
- Difficult to scale up to large projects where documentation.


2. Identify Stakeholders' win conditions
3. Reconcile win conditions. Establish next level objectives, constraints, alternatives
1. Identify next-level Stakeholders
4. Evaluate product and process alternatives. Resolve Risks
7. Review, commitment
6. Validate product and process definitions
5. Define next level of product and process - including partitions

Incremental Process Models

The incremental model combines elements of linear and parallel process flows
The incremental model applies linear sequences in a staggered fashion as calendar time progresses. Each linear sequence produces deliverable “increments” of the software [McD93] in a manner that is sim ilar to the increments produced by an evolutionary process flow .
The incremental process model focuses on the delivery of an operational product with each increment. Early increments are stripped-down versions of the final prod uct, but they do provide capability that serves the user and also provide a platform for evaluation by the user



Incremental development is particularly useful when staffing is unavailable for a complete implementation by the business deadline that has been established for the project

Advantages of incremental model –

- This model is flexible because the cost of development is low and initial product delivery is faster.
- It is easier to test and debug during the smaller iteration.
- The working software generates quickly and early during the software life cycle.
- The customers can respond to its functionalities after every increment.

Disadvantages of the incremental model-

- The cost of the final product may cross the cost estimated initially.
- This model requires a very clear and complete planning.
- The planning of design is required before the whole system is broken into small increments.
- The demands of customer for the additional functionalities after every increment causes problem during the system architecture.

Evolutionary Process Models-

Evolutionary models are iterative type models. They allow to develop more complete versions of the software.

Following are the evolutionary process models.

1. The prototyping model
2. The spiral model
3. Concurrent development model

The Prototyping model

- Prototype is defined as first or preliminary form using which other forms are copied or derived.
- Prototype model is a set of general objectives for software
- It does not identify the requirements like detailed input, output.
- It is software working model of limited functionality
- In this model, working programs are quickly produced.

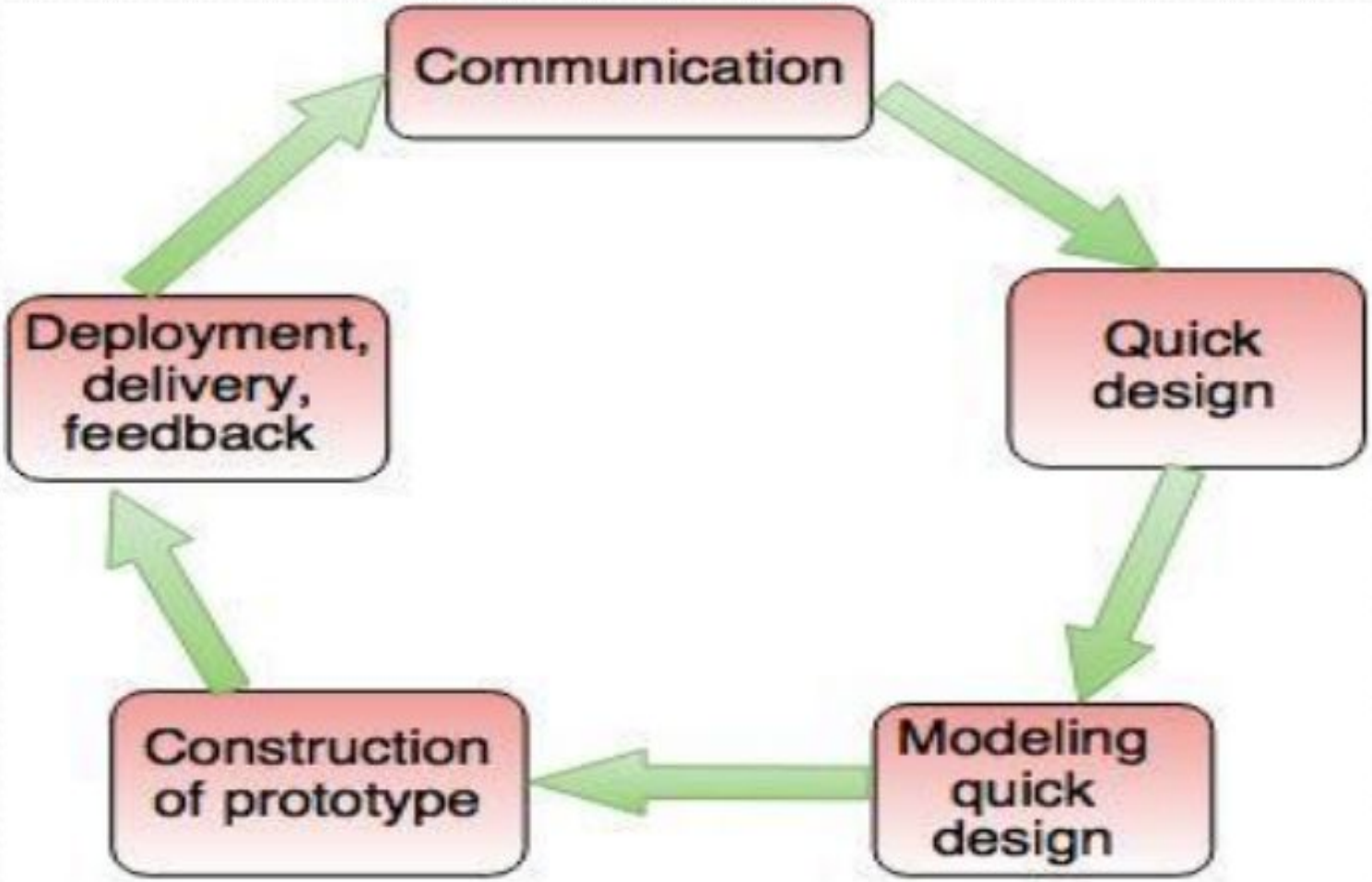


Fig. - The Prototyping Model

The different phases of Prototyping model are:

1. Communication-

In this phase, developer and customer meet and discuss the overall objectives of the software.

2. Quick design

Quick design is implemented when requirements are known. It includes only the important aspects like input and output format of the software. It focuses on those aspects which are visible to the user rather than the detailed plan. It helps to construct a prototype.

3. Modeling quick design

This phase gives the clear idea about the development of software because the software is now built. It allows the developer to better understand the exact requirements.

4. Construction of prototype The prototype is evaluated by the customer itself.

5. Deployment, delivery, feedback

If the user is not satisfied with current prototype then it refines according to the requirements of the user.

The process of refining the prototype is repeated until all the requirements of users are met. When the users are satisfied with the developed prototype then the system is developed on the basis of final prototype.

AGILE DEVELOPMENT

Agile software engi neering combines a philosophy and a set of development guidelines. The philosophy encourages customer satisfaction and early incremental delivery of software; small, highly motivated project teams; informal methods; minimal software engineer ing work products; and overall development simplicity.

Agility has become today's buzzword when describing a modern software process. Every one is agile. An agile team is a nimble team able to appropriately respond to changes. Change is what software development is very much about. Changes in the software be ing built, changes to the team members, changes because of new technology, changes of all kinds that may have an impact on the product they build or the project that creates the product.

Agility Principles-

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must work together daily throughout the project.

5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.

6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

9. Continuous attention to technical excellence and good design enhances agility.

10. Simplicity—the art of maximizing the amount of work not done—is essential.

11. The best architectures, requirements, and designs emerge from self organizing teams.

12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Introduction The DevOps is the combination of two words, one is Development and other is Operations. It is a culture to promote the development and operation process collectively.

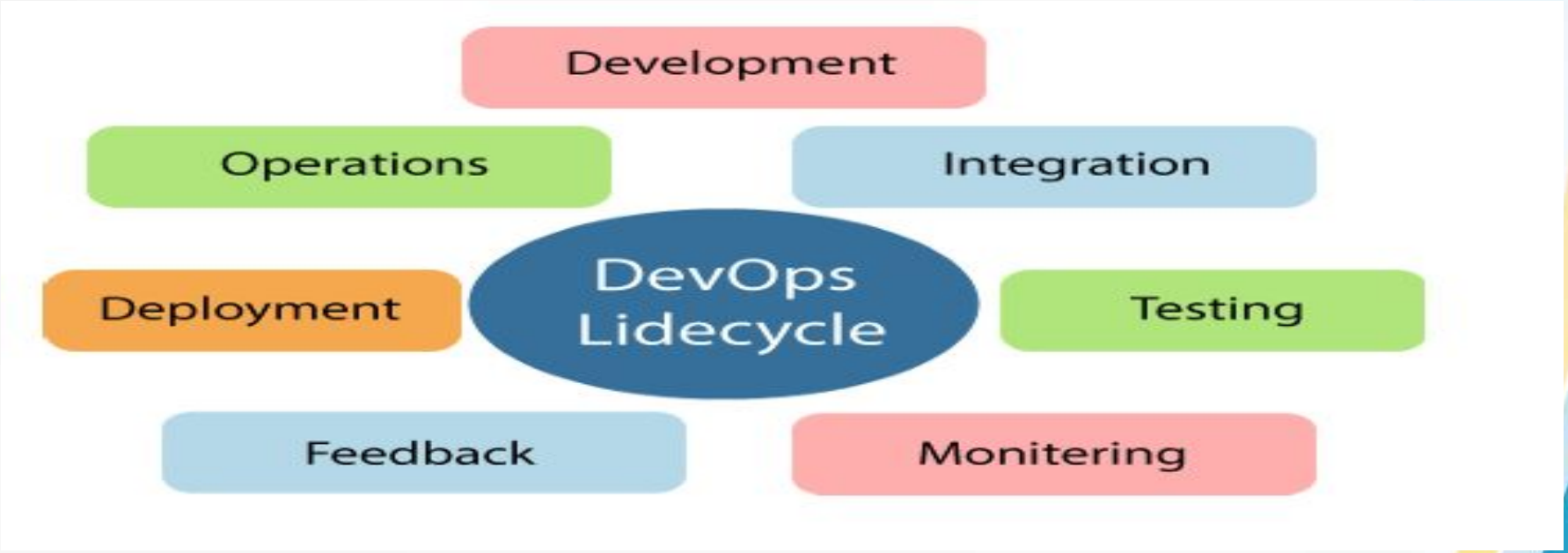
What is DevOps?

Why DevOps?

Before going further, we need to understand why we need the DevOps over the other methods. o The operation and development team worked in complete isolation. o After the design-build, the testing and deployment are performed respectively. That's why they consumed more time than actual build cycles.

DevOps Lifecycle

DevOps defines an agile relationship between operations and Development. It is a process that is practiced by the development team and operational engineers together from beginning to the final stage of the product.



Learning DevOps is not complete without understanding the DevOps lifecycle phases. The DevOps lifecycle includes seven phases as given below:

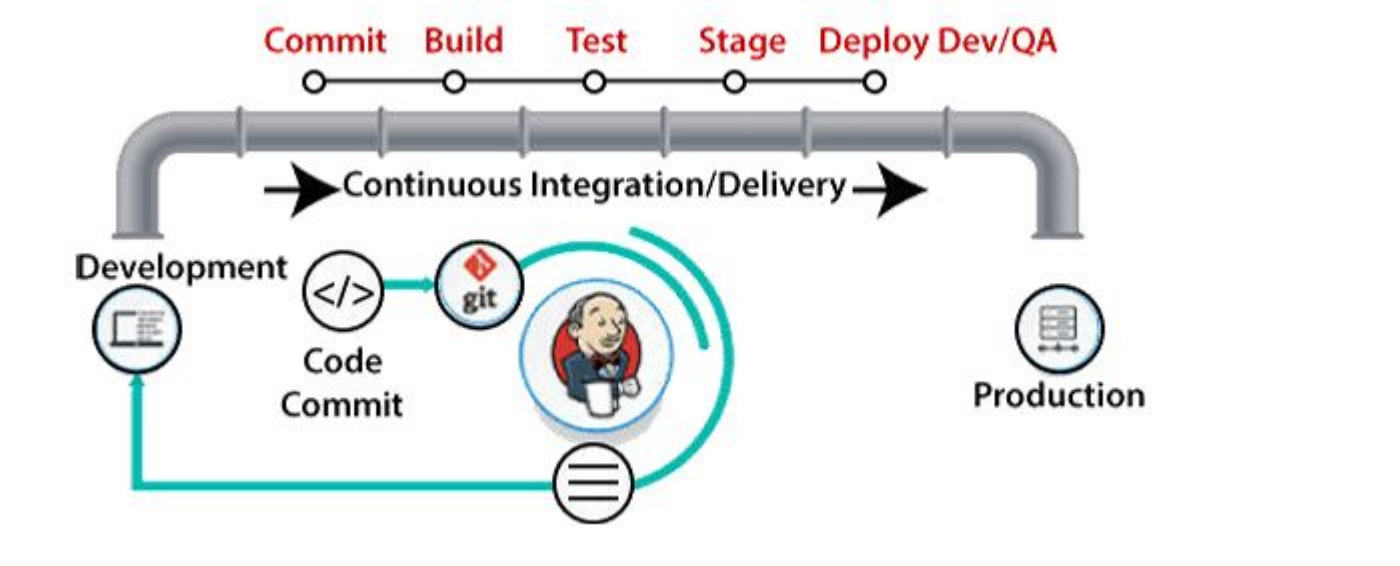
1) Continuous Development

This phase involves the planning and coding of the software. The vision of the project is decided during the planning phase. And the developers begin developing the code for the application. There

are no DevOps tools that are required for planning, but there are several tools for maintaining the code.

2) Continuous Integration This stage is the heart of the entire DevOps lifecycle. It is a software development practice in which the developers require to commit changes to the source code more frequently. This may be on a daily or weekly basis. Then every commit is built, and this allows early detection of problems if they are present. Building code is not only involved compilation, but it also includes unit testing, integration testing, code review, and packaging.

The code supporting new functionality is continuously integrated with the existing code. Therefore, there is continuous development of software. The updated code needs to be integrated continuously and smoothly with the systems to reflect changes to the end-users.



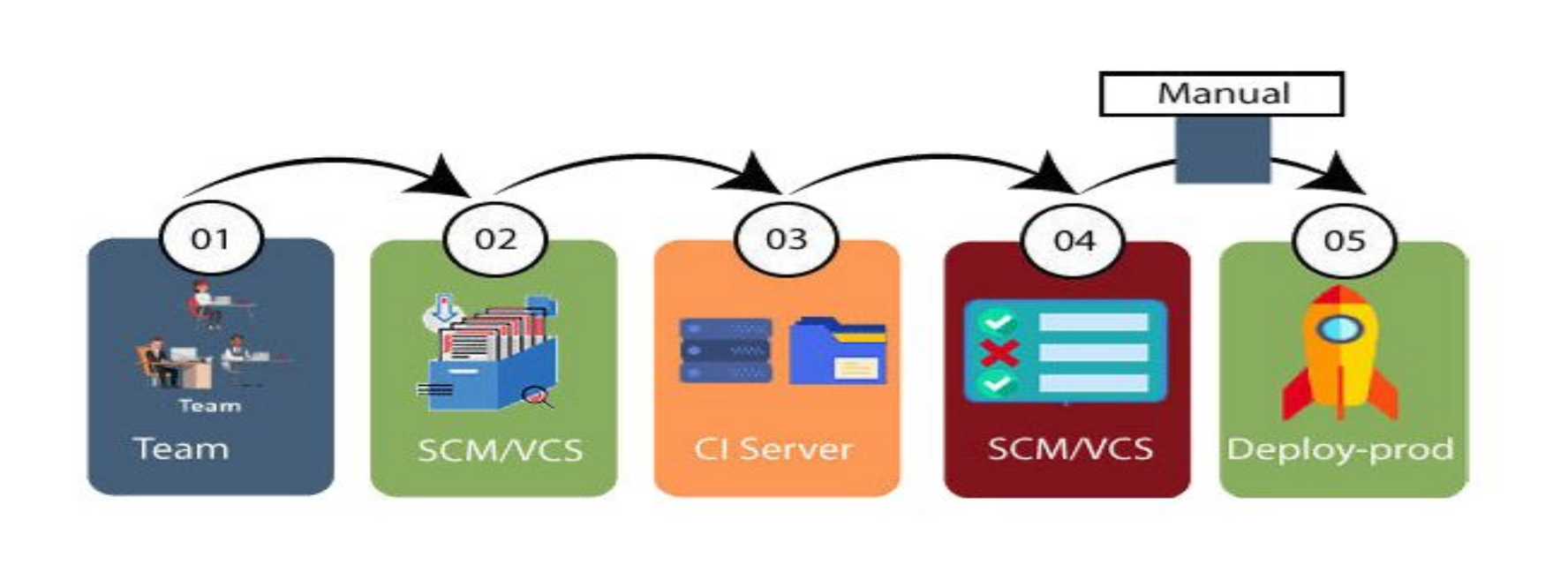
Jenkins is a popular tool used in this phase. Whenever there is a change in the Git repository, then Jenkins fetches the updated code and prepares a build of that code, which is an executable file in the form of war or jar. Then this build is forwarded to the test server or the production server.

3) Continuous Testing- This phase, where the developed software is continuously testing for bugs. For constant testing, automation testing tools such as TestNG, JUnit, Selenium, etc are used. These tools allow QAs to test multiple code-bases thoroughly in parallel to ensure that there is no flaw in the functionality. In this phase, Docker Containers can be used for simulating the test environment.

4) Continuous Monitoring- Monitoring is a phase that involves all the operational factors of the entire DevOps process, where important information about the use of the software is recorded and carefully processed to find out trends and identify problem areas. It may occur in the form of documentation files or maybe produce large-scale data about the application parameters when it is in a continuous use position. The system errors such as server not reachable, low memory, etc are resolved in this phase. It maintains the security and availability of the service.

5) Continuous Feedback- The application development is consistently improved by analyzing the results from the operations of the software. This is carried out by placing the critical phase of constant feedback between the operations and the development of the next version of the current software application. The continuity is the essential factor in the DevOps as it removes the unnecessary steps which are required to take a software application from development, using it to find out its issues and then

6) Continuous Deployment In this phase, the code is deployed to the production servers. Also, it is essential to ensure that the code is correctly used on all the servers.



Waterfall vs Agile vs DevOps

demandsimplified.marketing

SDLC	Waterfall	Agile	DevOps
Approach	Waterfall model provides a linear sequential approach to managing software projects. Each phase depends on deliverables from the previous one.	Agile is a highly dynamic and iterative approach where you do not need the complete set of requirements to start with. You can develop some features and check customer response before next steps	DevOps is an Agile methodology encompassing Development (Dev) and Operations (Ops). It enables end-to-end lifecycle delivery of features, fixes, and updates at frequent intervals.
Year	1970	2001	2009
Scope and Schedule	Adjust schedule to preserve scope. Fixed requirements. Limited flexibility.	Adjust scope to preserve schedule. Iterative approach allows for prioritization	Adjust scope to preserve schedule. Highly responsive to business needs
Time to Market	Slow	Rapid	Continuous
Automation	Low	Varied	High
Customer Feedback	End of Project	After every sprint	Continuous
Quality	Low. Issues are identified only at testing stage	Better. Issues are identified after every sprint	High. Automated unit testing during development
Collaboration	Teams operate in silos	Multiple teams are involved, but not all	All stakeholders are involved from start to finish
Variations	V-model	Scrum, XP, LeSS, SAFe	CI/CD, DevSecOps